Car Price Regression Analysis

Author: Amirah Shayim Mohammad Alreshidi.

Date: November 3, 2025

Project: AI Bootcamp Project.

Step 1: Import Libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import r2_score, mean_squared_error

print("Libraries imported successfully")
```

```
Libraries imported successfully
```

Step 2: Load and Inspect Data

```
df = pd.read_csv('/content/used_cars.csv')

print("--- First 5 Rows (Original Data) ---")
print(df.head())

print("\n--- Column Info (Original Data) ---")
df.info()
```

```
--- First 5 Rows (Original Data) ---
      brand                           model  model_year      milage  \
0      Ford  Utility Police Interceptor Base        2013  51,000 mi.
1   Hyundai                    Palisade SEL        2021  34,742 mi.
2     Lexus                    RX 350 RX 350        2022  22,372 mi.
3  INFINITI                 Q50 Hybrid Sport        2015  88,900 mi.
4      Audi        Q3 45 S line Premium Plus        2021   9,835 mi.

       fuel_type                                           engine  \
0  E85 Flex Fuel  300.0HP 3.7L V6 Cylinder Engine Flex Fuel Capa...
1       Gasoline                                3.8L V6 24V GDI DOHC
2       Gasoline                                      3.5 Liter DOHC
3         Hybrid  354.0HP 3.5L V6 Cylinder Engine Gas/Electric H...
4       Gasoline                         2.0L I4 16V GDI DOHC Turbo

        transmission                 ext_col int_col  \
0        6-Speed A/T                   Black   Black
1  8-Speed Automatic         Moonlight Cloud    Gray
2          Automatic                    Blue   Black
3        7-Speed A/T                   Black   Black
4  8-Speed Automatic  Glacier White Metallic   Black

                                accident clean_title    price
0  At least 1 accident or damage reported         Yes  $10,300
1  At least 1 accident or damage reported         Yes  $38,005
2                          None reported         NaN  $54,598
3                          None reported         Yes  $15,500
4                          None reported         NaN  $34,999

--- Column Info (Original Data) ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4009 entries, 0 to 4008
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   brand         4009 non-null   object
 1   model         4009 non-null   object
 2   model_year    4009 non-null   int64
 3   milage        4009 non-null   object
 4   fuel_type     3839 non-null   object
 5   engine        4009 non-null   object
 6   transmission  4009 non-null   object
 7   ext_col       4009 non-null   object
 8   int_col       4009 non-null   object
```

```
 9   accident      3896 non-null   object
 10  clean_title   3413 non-null   object
 11  price         4009 non-null   object
dtypes: int64(1), object(11)
memory usage: 376.0+ KB
```

Step 3: Handle Missing Data

```
df['fuel_type'] = df['fuel_type'].fillna('Unknown')
df['accident'] = df['accident'].fillna('Unknown')
df['clean_title'] = df['clean_title'].fillna('Unknown')

# For engine, we can fill missing values with the most common one (mode)
df['engine'] = df['engine'].fillna(df['engine'].mode()[0])

print("\n--- Info After Handling Missing Data ---")
df.info()
```

```
--- Info After Handling Missing Data ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4009 entries, 0 to 4008
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   brand         4009 non-null   object
 1   model         4009 non-null   object
 2   model_year    4009 non-null   int64
 3   milage        4009 non-null   object
 4   fuel_type     4009 non-null   object
 5   engine        4009 non-null   object
 6   transmission  4009 non-null   object
 7   ext_col       4009 non-null   object
 8   int_col       4009 non-null   object
 9   accident      4009 non-null   object
 10  clean_title   4009 non-null   object
 11  price         4009 non-null   object
dtypes: int64(1), object(11)
memory usage: 376.0+ KB
```

Step 4: Data Cleaning (Price and Mileage)

```
df['price'] = df['price'].astype(str).str.replace('$', '')
df['price'] = df['price'].str.replace(',', '')
df['price'] = pd.to_numeric(df['price'], errors='coerce')

df['milage'] = df['milage'].astype(str).str.replace(' mi.', '')
df['milage'] = df['milage'].str.replace(',', '')
df['milage'] = pd.to_numeric(df['milage'], errors='coerce')

df.dropna(subset=['price', 'milage'], inplace=True)

print("\n--- First 5 Rows (After Cleaning) ---")
print(df.head())

print("\n--- Column Info (After Cleaning) ---")
df.info()
```

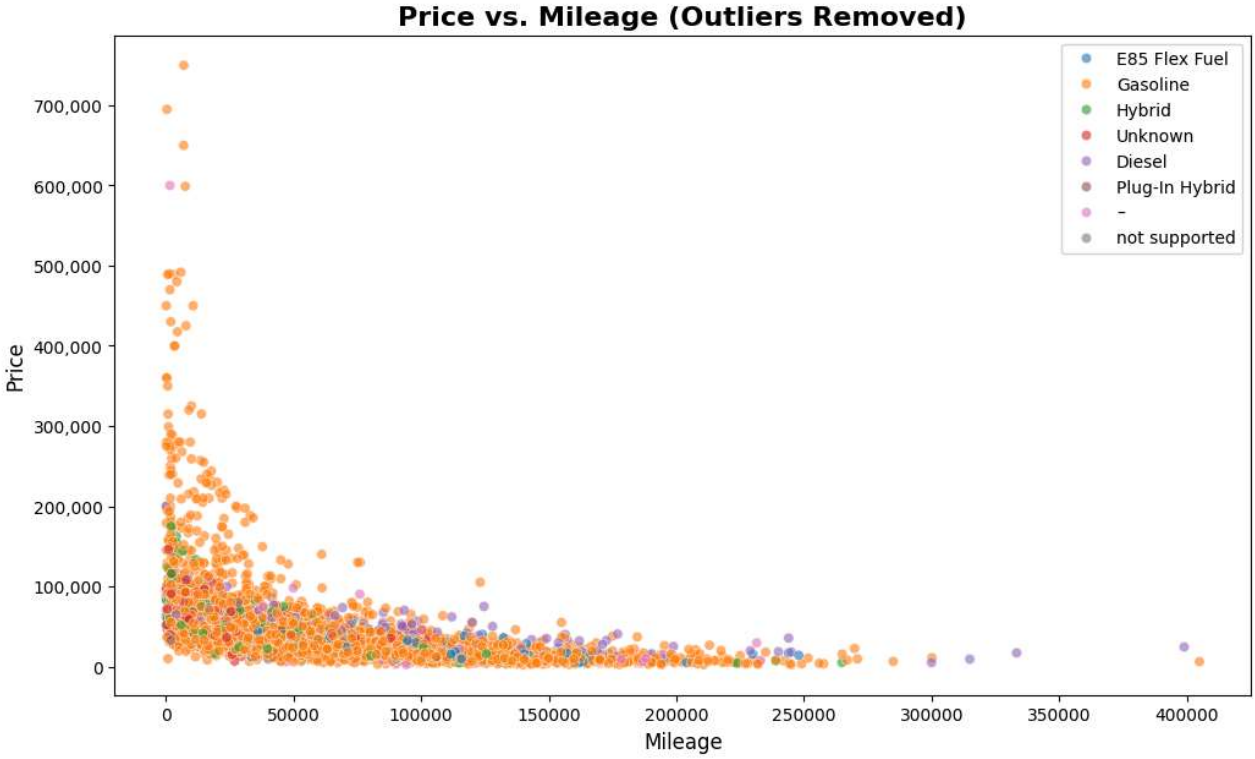
```
--- First 5 Rows (After Cleaning) ---
      brand                           model  model_year  milage  \
0      Ford  Utility Police Interceptor Base        2013   51000
1   Hyundai                    Palisade SEL        2021   34742
2     Lexus                   RX 350 RX 350        2022   22372
3  INFINITI                Q50 Hybrid Sport        2015   88900
4      Audi        Q3 45 S line Premium Plus        2021    9835

       fuel_type                                          engine  \
0  E85 Flex Fuel  300.0HP 3.7L V6 Cylinder Engine Flex Fuel Capa...
1       Gasoline                                3.8L V6 24V GDI DOHC
2       Gasoline                                      3.5 Liter DOHC
3         Hybrid  354.0HP 3.5L V6 Cylinder Engine Gas/Electric H...
4       Gasoline                         2.0L I4 16V GDI DOHC Turbo

        transmission                ext_col int_col  \
0        6-Speed A/T                  Black   Black
1  8-Speed Automatic         Moonlight Cloud    Gray
2          Automatic                   Blue   Black
3        7-Speed A/T                  Black   Black
4  8-Speed Automatic  Glacier White Metallic   Black
```

```
                                  accident clean_title  price
0  At least 1 accident or damage reported         Yes  10300
1  At least 1 accident or damage reported         Yes  38005
2                          None reported     Unknown  54598
3                          None reported         Yes  15500
4                          None reported     Unknown  34999

--- Column Info (After Cleaning) ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4009 entries, 0 to 4008
Data columns (total 12 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   brand         4009 non-null   object
 1   model         4009 non-null   object
 2   model_year    4009 non-null   int64
 3   milage        4009 non-null   int64
 4   fuel_type     4009 non-null   object
 5   engine        4009 non-null   object
 6   transmission  4009 non-null   object
 7   ext_col       4009 non-null   object
 8   int_col       4009 non-null   object
 9   accident      4009 non-null   object
 10  clean_title   4009 non-null   object
 11  price         4009 non-null   int64
dtypes: int64(3), object(9)
memory usage: 376.0+ KB
```

Step 5: Remove Price Outliers

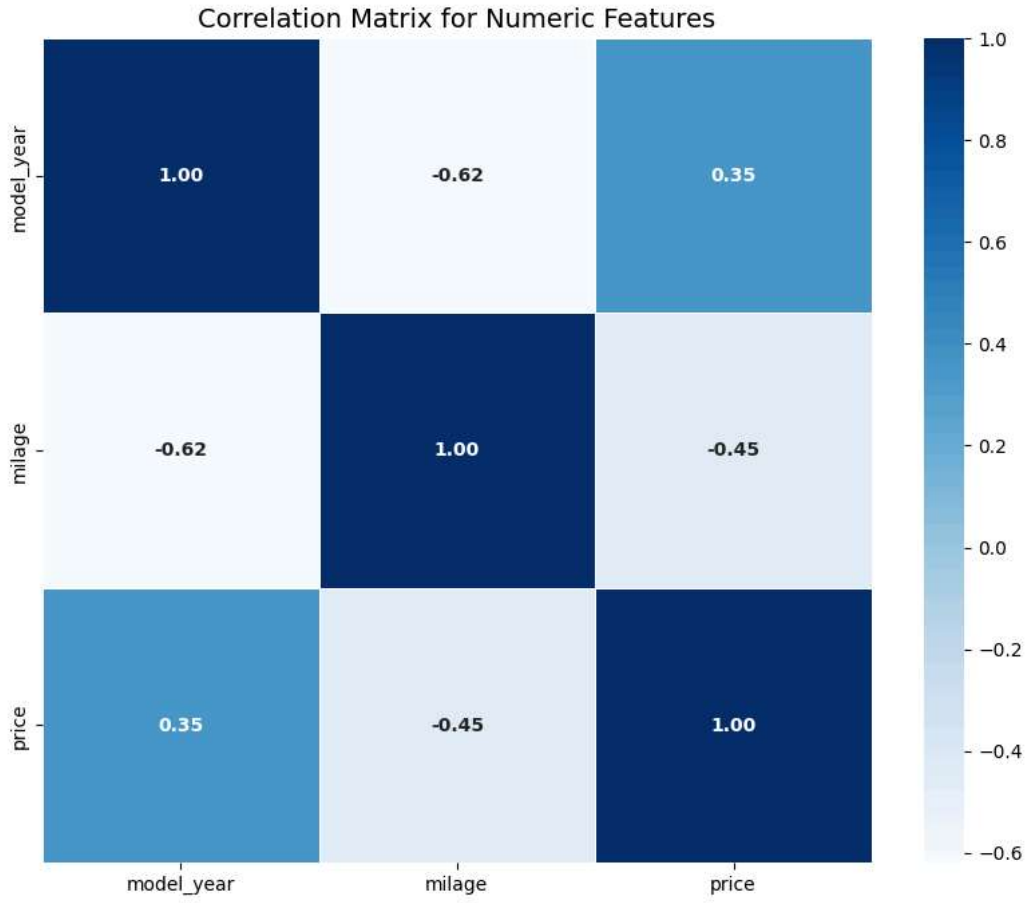
```
print(f"Original data shape: {df.shape}")

df = df[df['price'] < 1000000]

print(f"New data shape after removing outliers: {df.shape}")

print("\nPlotting Price vs. Mileage (Outliers Removed):")
plt.figure(figsize=(12, 7))
sns.scatterplot(data=df, x='milage', y='price', hue='fuel_type', alpha=0.6)
plt.title('Price vs. Mileage (Outliers Removed)', weight='bold', fontsize=16)
plt.xlabel('Mileage', fontsize=12)
plt.ylabel('Price', fontsize=12)
plt.gca().get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p: format(int(x), ',')))
plt.legend(loc='best', fontsize=10)
plt.show()
```

```
Original data shape: (4009, 12)
New data shape after removing outliers: (4006, 12)

Plotting Price vs. Mileage (Outliers Removed):
```



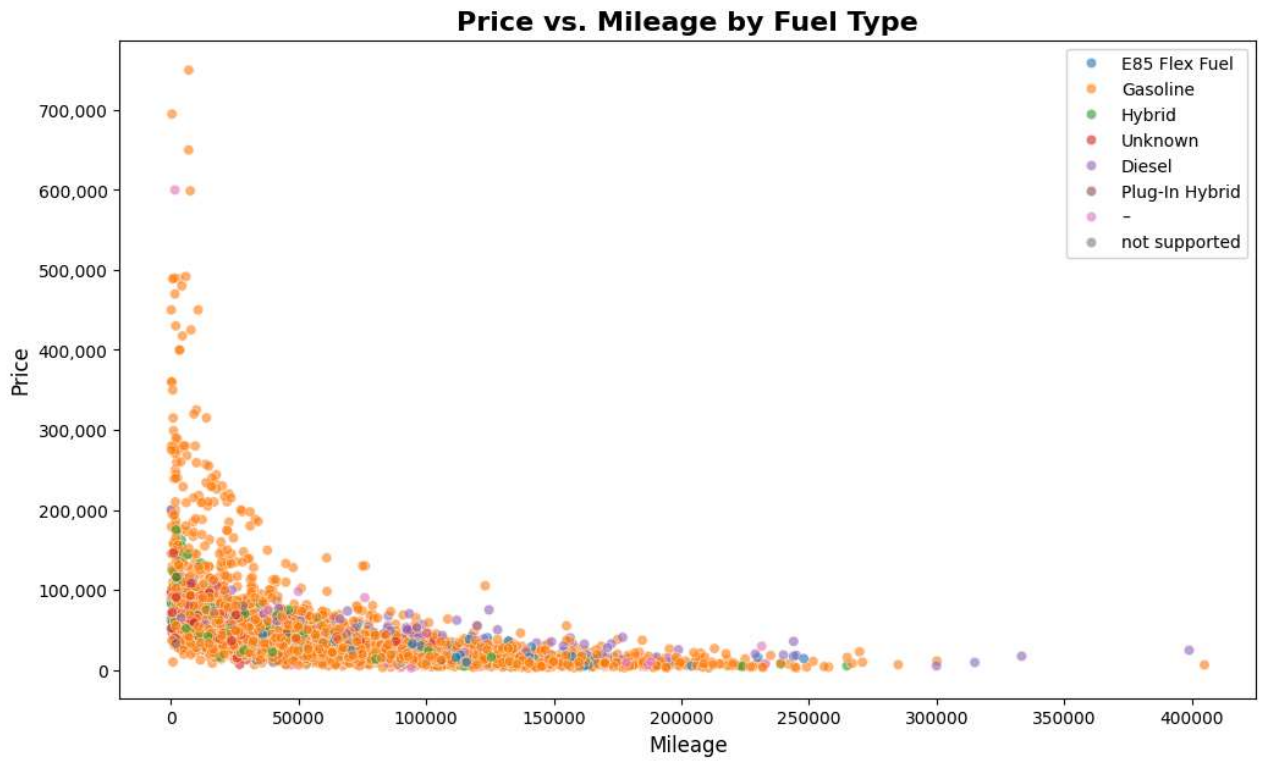
Step 6: EDA - Correlation Matrix

```
numeric_df = df.select_dtypes(include=['number'])
correlation_matrix = numeric_df.corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='Blues', linewidths=0.5, fmt='.2f',
            annot_kws={'fontsize': 10, 'fontweight': 'bold'})
plt.title('Correlation Matrix for Numeric Features', fontsize=14)
plt.show()
```



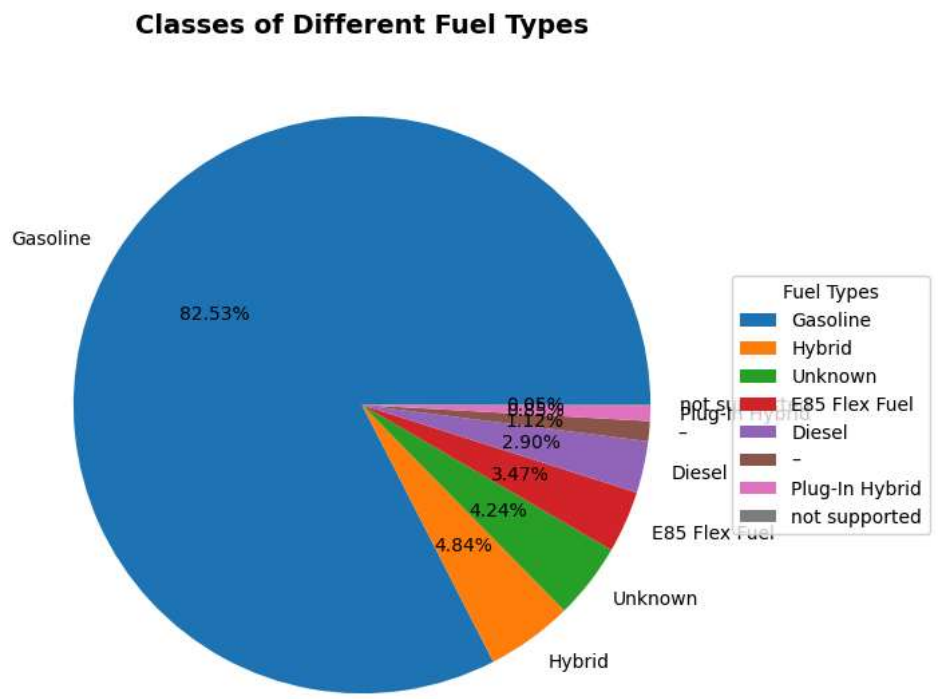
Step 7: EDA - Price vs. Mileage Scatter Plot

```
plt.figure(figsize=(12, 7))
sns.scatterplot(data=df, x='milage', y='price', hue='fuel_type', alpha=0.6)
plt.title('Price vs. Mileage by Fuel Type', weight='bold', fontsize=16)
plt.xlabel('Mileage', fontsize=12)
plt.ylabel('Price', fontsize=12)
plt.gca().get_yaxis().set_major_formatter(plt.FuncFormatter(lambda x, p: format(int(x), ',')))
plt.legend(loc='best', fontsize=10)
plt.show()
```



Step 8: EDA - Fuel Type Distribution

```
plt.figure(figsize=(10, 7))
df['fuel_type'].value_counts().plot(kind='pie', autopct='%.2f%%')
plt.title('Classes of Different Fuel Types', weight='bold', fontsize=14)
plt.ylabel('')
plt.legend(title='Fuel Types', loc='center left', bbox_to_anchor=(1, 0, 0.5, 1))
plt.show()
```



Step 9: Feature Engineering (Engine)

```
print("\nStarting Feature Engineering for 'engine'...")

df['engine'] = df['engine'].astype(str).str.lower()

df['engine_V6'] = df['engine'].str.contains('v6').astype(int)
df['engine_V8'] = df['engine'].str.contains('v8').astype(int)
df['engine_I4'] = df['engine'].str.contains('i4|4-cyl').astype(int)
df['engine_Turbo'] = df['engine'].str.contains('turbo').astype(int)
df['engine_Hybrid'] = df['engine'].str.contains('hybrid').astype(int)
df['engine_Electric'] = df['engine'].str.contains('electric|battery').astype(int)

print("Feature Engineering complete. New 'engine_' columns added.")
print(df[['engine', 'engine_V6', 'engine_V8', 'engine_I4']].head())
```

```
Starting Feature Engineering for 'engine'...
Feature Engineering complete. New 'engine_' columns added.
                                              engine  engine_V6  engine_V8  \
0  300.0hp 3.7l v6 cylinder engine flex fuel capa...          1          0
1                                3.8l v6 24v gdi dohc          1          0
2                                      3.5 liter dohc          0          0
3  354.0hp 3.5l v6 cylinder engine gas/electric h...          1          0
4                          2.0l i4 16v gdi dohc turbo          0          0

   engine_I4
0          0
1          0
2          0
3          0
4          1
```

Step 10: Data Preprocessing (Encoding)

```
from sklearn.preprocessing import LabelEncoder

df_model = df.copy()

le = LabelEncoder()
df_model['model'] = le.fit_transform(df_model['model'])
df_model['ext_col'] = le.fit_transform(df_model['ext_col'])
df_model['int_col'] = le.fit_transform(df_model['int_col'])


categorical_cols = ['fuel_type', 'transmission', 'accident', 'clean_title', 'brand']

df_model = pd.get_dummies(df_model, columns=categorical_cols, drop_first=True, dtype=int)

print("--- Data is now ready for the model (LabelEncoder + get_dummies applied) ---")
print(df_model.head())
```

```
--- Data is now ready for the model (LabelEncoder + get_dummies applied) ---
   model  model_year  milage  \
0   1742        2013   51000
1   1181        2021   34742
2   1324        2022   22372
3   1241        2015   88900
4   1224        2021    9835

                                              engine  ext_col  int_col  price  \
0  300.0hp 3.7l v6 cylinder engine flex fuel capa...       29       14  10300
1                                3.8l v6 24v gdi dohc      185       71  38005
2                                      3.5 liter dohc       38       14  54598
3  354.0hp 3.5l v6 cylinder engine gas/electric h...       29       14  15500
4                          2.0l i4 16v gdi dohc turbo      120       14  34999

   engine_V6  engine_V8  engine_I4  ...  brand_Saab  brand_Saturn  \
0          1          0          0  ...           0             0
1          1          0          0  ...           0             0
2          0          0          0  ...           0             0
3          1          0          0  ...           0             0
4          0          0          1  ...           0             0

   brand_Scion  brand_Subaru  brand_Suzuki  brand_Tesla  brand_Toyota  \
0            0             0             0            0             0
1            0             0             0            0             0
2            0             0             0            0             0
3            0             0             0            0             0
4            0             0             0            0             0

   brand_Volkswagen  brand_Volvo  brand_smart
0                 0            0            0
1                 0            0            0
2                 0            0            0
3                 0            0            0
```

```
4                0            0            0

[5 rows x 139 columns]
```

Step 11: Define Features (X) and Target (y)

```
y = df_model['price']

X = df_model.drop(['price', 'engine'], axis=1)

X = X.select_dtypes(include=['number'])

print(f"Features (X) shape: {X.shape}")
print("--- Example of processed data (X) ---")
print(X.head())
```

```
Features (X) shape: (4006, 137)
--- Example of processed data (X) ---
   model  model_year  milage  ext_col  int_col  engine_V6  engine_V8  \
0   1742        2013   51000       29       14          1          0
1   1181        2021   34742      185       71          1          0
2   1324        2022   22372       38       14          0          0
3   1241        2015   88900       29       14          1          0
4   1224        2021    9835      120       14          0          0

   engine_I4  engine_Turbo  engine_Hybrid  ...  brand_Saab  brand_Saturn  \
0          0             0              0  ...           0             0
1          0             0              0  ...           0             0
2          0             0              0  ...           0             0
3          0             0              1  ...           0             0
4          1             1              0  ...           0             0

   brand_Scion  brand_Subaru  brand_Suzuki  brand_Tesla  brand_Toyota  \
0            0             0             0            0             0
1            0             0             0            0             0
2            0             0             0            0             0
3            0             0             0            0             0
4            0             0             0            0             0

   brand_Volkswagen  brand_Volvo  brand_smart
0                 0            0            0
1                 0            0            0
2                 0            0            0
3                 0            0            0
4                 0            0            0

[5 rows x 137 columns]
```

Step 12: Split Data into Training and Testing Sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

print(f"Training data (X_train): {X_train.shape}")
print(f"Testing data (X_test): {X_test.shape}")
```

```
Training data (X_train): (2804, 137)
Testing data (X_test): (1202, 137)
```

Step 13: Train Model 1 - Linear Regression

```
model = LinearRegression()
model.fit(X_train, y_train)

print("Model 1 (Linear Regression) training complete!")
```

```
Model 1 (Linear Regression) training complete!
```

Step 14: Evaluate Model 1 (Linear Regression)

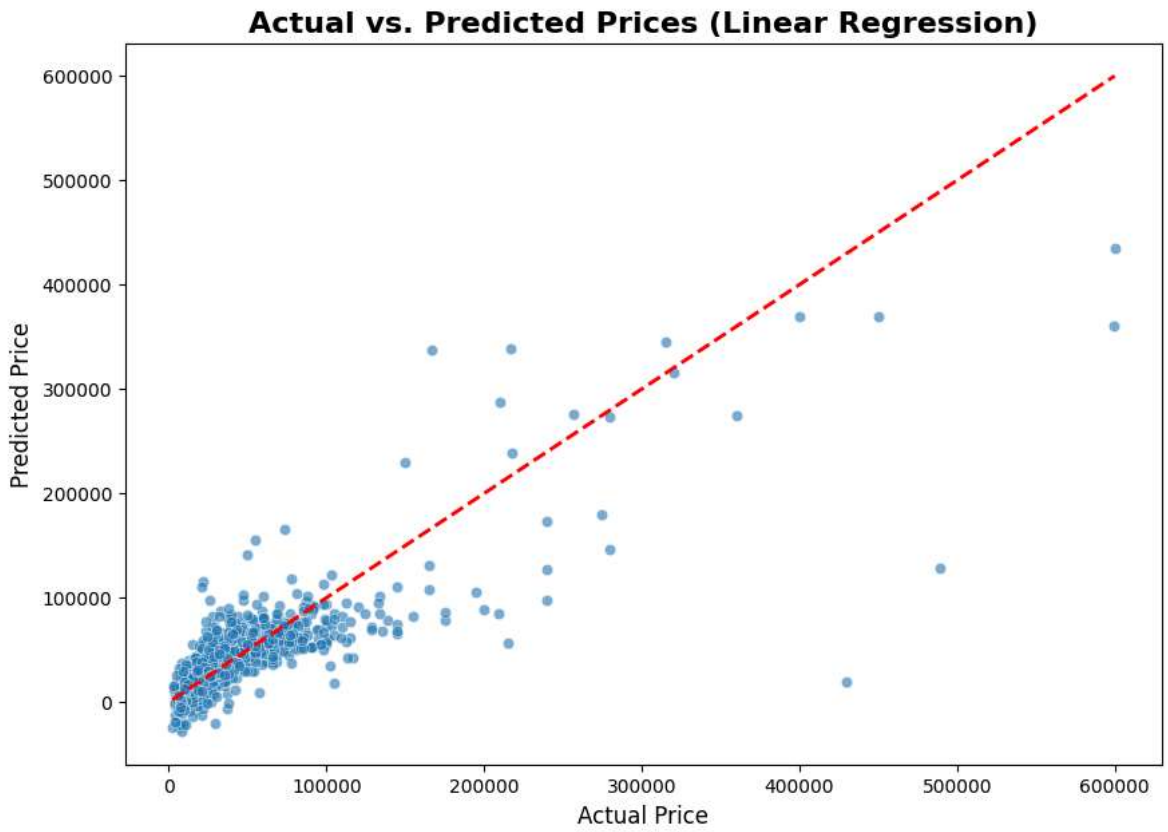
```
y_pred = model.predict(X_test)

r2_lr = r2_score(y_test, y_pred)
rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred))

print(f"--- Model 1 (Linear Regression) Performance ---")
```

```
print(f"R-squared (R²): {r2_lr:.4f}")
print(f"Root Mean Squared Error (RMSE): ${rmse_lr:,.2f}")
```

```
--- Model 1 (Linear Regression) Performance ---
R-squared (R²): 0.6542
Root Mean Squared Error (RMSE): $29,102.29
```

Step 15: Visualize Model 1 Results

```
plt.figure(figsize=(10, 7))
sns.scatterplot(x=y_test, y=y_pred, alpha=0.6)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2)
plt.title('Actual vs. Predicted Prices (Linear Regression)', fontsize=16, weight='bold')
plt.xlabel('Actual Price', fontsize=12)
plt.ylabel('Predicted Price', fontsize=12)
plt.show()
```



Step 16: Train & Evaluate Model 2 - Random Forest

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42, n_jobs=-1) # n_jobs=-1 makes it train faster

print("\nStarting Random Forest model training...")
rf_model.fit(X_train, y_train)
print("Model 2 (Random Forest) training complete!")
```

```
Starting Random Forest model training...
Model 2 (Random Forest) training complete!
```

```
y_pred_rf = rf_model.predict(X_test)

r2_rf = r2_score(y_test, y_pred_rf)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

print(f"--- Model 2 (Random Forest) Performance ---")
print(f"R-squared (R²): {r2_rf:.4f}")
print(f"Root Mean Squared Error (RMSE): ${rmse_rf:,.2f}")
```